



Per-CPU Maps for Lockless Metrics

Lesson 08: Microsecond-Accurate Packet and Byte Counting
Under High Concurrency

Table of Contents

Lesson 08: Per-CPU Maps for Lockless Metrics

1. Lesson Overview
2. Foundations & Core Technical Concepts
 - The Problem: Cache-Line Bouncing
 - What is a CPU Cache Line?
 - How BPF_MAP_TYPE_PERCPU_ARRAY Solves This
 - How Per-CPU Map Memory is Organized
 - PERCPU_ARRAY vs PERCPU_HASH
3. Problem Statement & Real-World Use Case
 - The Problem: Shared Counters at 10Gbps
 - Real Production Story: Meta's Katran Load Balancer
4. Cache-Line Geometry Visualization
5. Architecture Diagram
6. Per-CPU Update Sequence
7. Map Memory Layout
8. Step-by-Step Code Walkthrough
 - The eBPF C Program (main.bpf.c)
 - The Go Loader (main.go)
9. Running the Lab
10. Debugging
11. Common Mistakes
 - Mistake: Using atomic ops on per-CPU maps
 - Mistake: Incorrect per-CPU lookup in Go
12. Performance Analysis
13. Best Practices
14. Summary
15. Further Reading

Technical Deep-Dive Q&A: Per-CPU Maps for Lockless Metrics

1. Beginner Level
 - Q1. Why does updating a standard BPF_MAP_TYPE_ARRAY from multiple CPU cores simultaneously cause performance issues?
 - Q2. How does BPF_MAP_TYPE_PERCPU_ARRAY solve cache-line bouncing?
2. Intermediate Level

Q3. Why do we not need atomic operations (like `__sync_fetch_and_add()`) when updating a per-CPU map?

Q4. Walk through the memory allocation layout of a `BPF_MAP_TYPE_PERCPU_ARRAY`. How is alignment handled?

3. Advanced Level

Q5. How does Go retrieve and aggregate data from a `PERCPU_ARRAY` map?

4. System Design

Q6. Design a lockless packet counter for a high-performance DDoS mitigation system processing 40M packets/sec.

5. Troubleshooting

Q7. Why does my Go loader fail with invalid argument when doing a Lookup on a per-CPU map?

Hands-on Lab & Homework Assignments

1. Practical Exercises

Task 1: Check System Possible CPUs

Task 2: Observe Per-CPU Metrics

2. Coding Assignment: Per-CPU Protocol Breakdown

Requirements:

Lesson 08: Per-CPU Maps for Lockless Metrics

1. Lesson Overview

In this lesson, you will build a **zero-contention, microsecond-accurate packet metrics collector** using `BPF_MAP_TYPE_PERCPU_ARRAY`. You will understand why standard shared-memory counters fail at 10Gbps line rates due to CPU cache-line bouncing, and how per-CPU maps eliminate all lock contention by giving each CPU core its own isolated counter that is only aggregated in user-space.

This is the exact architecture used in Meta's **Katran** load balancer for tracking billions of packets per second per node.

2. Foundations & Core Technical Concepts

The Problem: Cache-Line Bouncing

Modern CPUs operate at 3–5 GHz, processing billions of instructions per second. However, accessing a shared memory location from multiple cores simultaneously triggers a hardware protocol called **MESI (Modified/Exclusive/Shared/Invalid)** cache coherence.

What happens when two cores update a shared counter:

1. CPU 0 reads the counter from DRAM into its L1 cache. L1 latency: ~4ns.
2. CPU 1 also reads the counter. Both caches have the value.
3. CPU 0 increments and writes back. Its cache line is now "Modified".
4. CPU 0 broadcasts a **cache invalidation** to all other cores.
5. CPU 1's cache entry for this counter is now "Invalid".
6. CPU 1 must re-fetch the counter from CPU 0's cache or DRAM. Cross-socket DRAM latency: ~60–100ns.
7. CPU 1 increments and writes back. Another invalidation broadcast.

At 10M packets/sec across 8 cores, this creates a **shared counter bottleneck** causing:

- Cache invalidation storms consuming the inter-core bus bandwidth.

- Each counter increment serializing across all cores.
- Effective packet processing rate limited to a fraction of theoretical maximum.

What is a CPU Cache Line?

A cache line is the unit of data transfer between DRAM and CPU cache — typically **64 bytes** on x86_64.

Key insight: Even if your counter is only 8 bytes (uint64), the CPU transfers and locks the entire 64-byte cache line containing it. If you have two counters in the same 64-byte region, updating one counter invalidates the cache line for both — even if the other counter is unrelated. This is called **false sharing**.

How BPF_MAP_TYPE_PERCPU_ARRAY Solves This

`BPF_MAP_TYPE_PERCPU_ARRAY` allocates **N separate copies of the map values** — one per CPU core. When CPU 3 increments index 0 in a per-CPU array, it writes to CPU 3's private copy:

- No cache line invalidation for CPUs 0, 1, 2.
- No MESI protocol overhead.
- The increment is a pure local memory write — ~1ns latency.

User-space aggregates the N copies by summing all per-CPU values when it needs the total.

How Per-CPU Map Memory is Organized

For a PERCPU_ARRAY with 1 entry (index 0) on a 4-core system:

Kernel allocates: 4 separate 64-byte aligned memory regions

```
CPU 0's copy: DRAM page at 0xffff_0000_0000 [u64: 0]
CPU 1's copy: DRAM page at 0xffff_0001_0000 [u64: 0]
CPU 2's copy: DRAM page at 0xffff_0002_0000 [u64: 0]
CPU 3's copy: DRAM page at 0xffff_0003_0000 [u64: 0]
```

Each copy is on a separate cache line, often on a separate NUMA node's memory. No cross-core traffic occurs during updates.

PERCPU_ARRAY vs PERCPU_HASH

Property	PERCPU_ARRAY	PERCPU_HASH
Key type	Integer index (u32)	Arbitrary hash key
Lookup speed	O(1) — direct index	O(1) — hash table
Memory allocation	Pre-allocated (all slots)	On-demand
Use case	Small, fixed counters (metrics)	Large, dynamic per-CPU state

Use `PERCPU_ARRAY` for fixed counters (accept/drop/error counts). Use `PERCPU_HASH` for per-source-IP per-CPU rate limiting.

3. Problem Statement & Real-World Use Case

The Problem: Shared Counters at 10Gbps

A single 10 Gbps NIC can deliver ~14.88 million minimum-size packets per second. If an XDP program uses a standard `BPF_MAP_TYPE_ARRAY` counter and 8 CPU cores are processing packets (via RSS — Receive Side Scaling), each core competes for the same 8-byte counter:

- **Theoretical max:** 14.88 Mpps across 8 cores = ~1.86 Mpps/core.
- **Reality with shared counter:** Lock contention on the counter serializes updates, reducing throughput to ~2–3 Mpps total — an 80% degradation.

Real Production Story: Meta's Katran Load Balancer

Katran is Meta's open-source XDP load balancer, processing millions of packets per second per host across thousands of nodes in Meta's data centers.

Katran's metrics architecture uses `BPF_MAP_TYPE_PERCPU_ARRAY` for:

- Per-backend packet and byte counters (tracking how many packets went to each backend server).
- Drop counters (packets dropped due to no healthy backend available).
- Error counters (malformed packets, unsupported protocols).

Their Go user-space agent periodically aggregates the per-CPU values into a Prometheus metrics endpoint. This runs on 40-core hosts at 40 Mpps with **zero lock contention** in the metrics path.

Cilium uses `BPF_MAP_TYPE_PERCPU_ARRAY` for its policy enforcement metrics — counting allowed and denied packets per network policy rule, per CPU.

4. Cache-Line Geometry Visualization

WITHOUT Per-CPU (`BPF_MAP_TYPE_ARRAY`):

Cache Line (64 bytes):

[counter_accept (8B) | counter_drop (8B) | ... padding (48B)]

CPU 0 updates counter_accept → invalidates cache line for ALL CPUs

CPU 1 updates counter_drop → must wait for CPU 0's update to propagate

CPU 2 updates counter_accept → must wait for CPU 1...

Result: Sequential execution of updates. BOTTLENECK.

WITH Per-CPU (`BPF_MAP_TYPE_PERCPU_ARRAY`):

CPU 0's copy: [counter_accept=1420 | counter_drop=3] ← on CPU 0's NUMA node

CPU 1's copy: [counter_accept= 890 | counter_drop=1] ← on CPU 1's NUMA node

CPU 2's copy: [counter_accept=2100 | counter_drop=7] ← on CPU 2's NUMA node

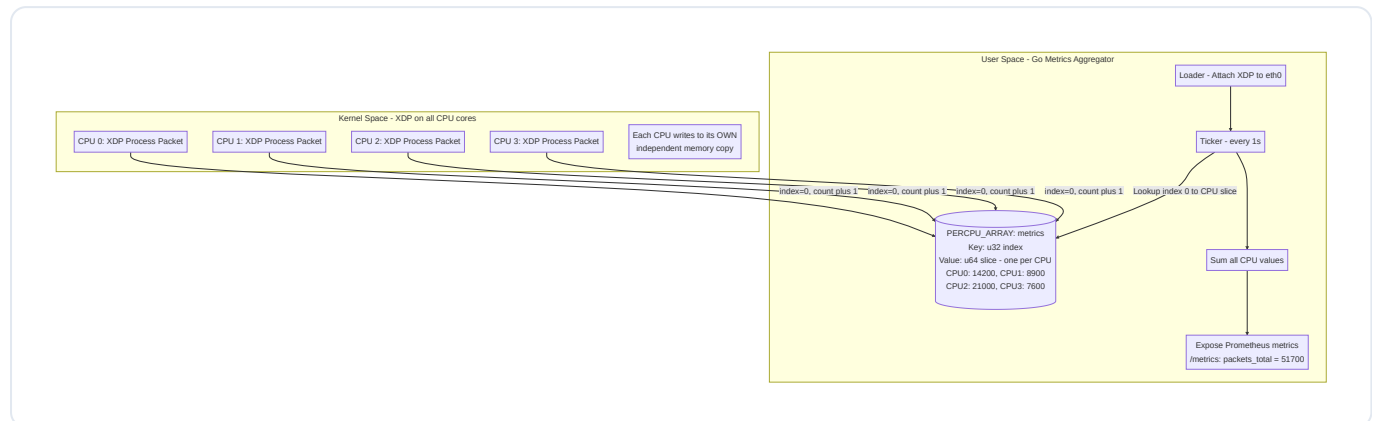
CPU 0, 1, 2 update simultaneously with ZERO contention.

User-space aggregation:

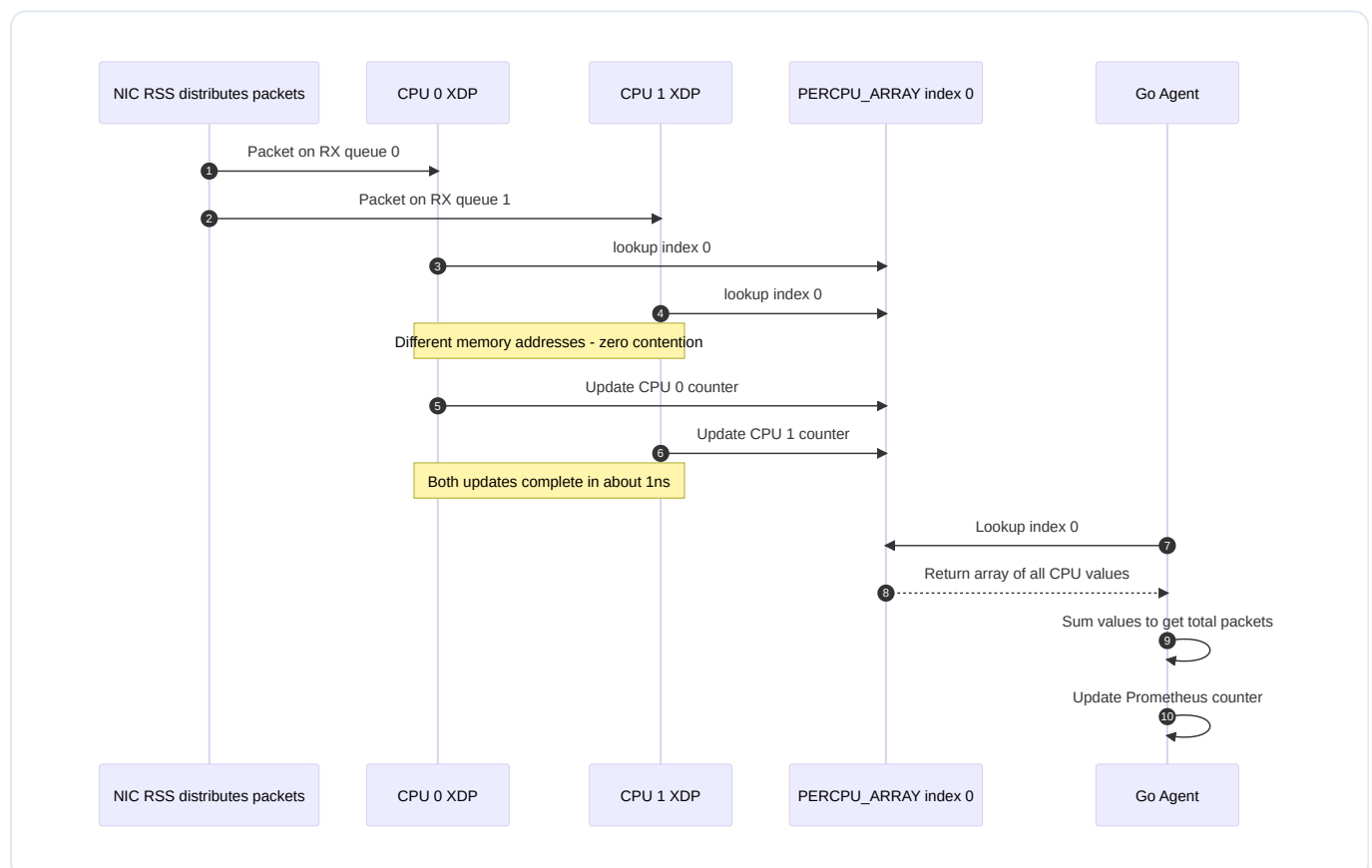
Total accept = 1420 + 890 + 2100 = 4410

Total drop = 3 + 1 + 7 = 11

5. Architecture Diagram



6. Per-CPU Update Sequence



7. Map Memory Layout

```
BPF_MAP_TYPE_PERCPU_ARRAY: metrics
max_entries: 3 (accept=0, drop=1, error=2)
num_possible_cpus: 4
```

Kernel memory layout (each on separate cache-aligned pages):

	Index 0 (accept)	Index 1 (drop)	Index 2 (error)	
CPU 0 slot:	[14,200	3	0]	(64B aligned)
CPU 1 slot:	[8,900	1	2]	(64B aligned)
CPU 2 slot:	[21,000	7	0]	(64B aligned)
CPU 3 slot:	[7,600	0	1]	(64B aligned)

Total memory: $3 \times 4 \times 8$ bytes = 96 bytes (but 4KB actual, due to alignment)

8. Step-by-Step Code Walkthrough

The eBPF C Program (`main.bpf.c`)

```
#include <vmlinux.h>
#include <bpf/bpf_helpers.h>
#include <bpf/bpf_endian.h>

// Metric indices – use enum for clarity
enum metric_index {
    METRIC_ACCEPT = 0,
    METRIC_DROP   = 1,
    METRIC_ERROR  = 2,
    METRIC_MAX     = 3,
};

// Per-CPU Array: each CPU has its own private copy of these counters
// No locks needed – each CPU only writes to its own copy
struct {
    __uint(type, BPF_MAP_TYPE_PERCPU_ARRAY);
    __uint(max_entries, METRIC_MAX);
    __type(key, __u32);
    __type(value, __u64);    // u64 counter per CPU per metric
} metrics SEC(".maps");

// Helper: increment a metric counter for the current CPU
static __always_inline void inc_metric(enum metric_index idx) {
    __u32 key = idx;
    __u64 *count = bpf_map_lookup_elem(&metrics, &key);
    if (count) {
        // This is a local write to the current CPU's private copy
        // NO ATOMIC NEEDED – per-CPU guarantees no concurrent writes to this copy
        *count += 1;
    }
}

SEC("xdp")
int xdp_metric_count(struct xdp_md *ctx) {
    void *data_end = (void *) (long) ctx->data_end;
    void *data = (void *) (long) ctx->data;

    // Validate minimum Ethernet frame size
    struct ethhdr *eth = data;
    if ((void *) (eth + 1) > data_end) {
```

```
        inc_metric(METRIC_ERROR);
        return XDP_PASS;
    }

    // Only count IPv4 traffic
    if (eth->h_proto != bpf_htons(ETH_P_IP)) {
        return XDP_PASS;    // Not counted – not an error, just not IPv4
    }

    struct iphdr *iph = (void *)(eth + 1);
    if ((void *) (iph + 1) > data_end) {
        inc_metric(METRIC_ERROR);
        return XDP_PASS;
    }

    // Count all IPv4 packets as accepted
    inc_metric(METRIC_ACCEPT);
    return XDP_PASS;
}

char LICENSE[] SEC("license") = "GPL";
```

The Go Loader (`main.go`)

```
package main

import (
    "fmt"
    "log"
    "os"
    "os/signal"
    "runtime"
    "syscall"
    "time"

    "github.com/cilium/ebpf"
    "github.com/cilium/ebpf/link"
)

const (
    MetricAccept = 0
    MetricDrop   = 1
    MetricError  = 2
)

func main() {
    spec, err := ebpf.LoadCollectionSpec("main.bpf.o")
    if err != nil {
        log.Fatalf("load spec: %v", err)
    }

    var objs struct {
        Prog      *ebpf.Program `ebpf:"xdp_metric_count"`
        Metrics *ebpf.Map      `ebpf:"metrics"`
    }
    if err := spec.LoadAndAssign(&objs, nil); err != nil {
        log.Fatalf("load objects: %v", err)
    }
    defer objs.Prog.Close()
    defer objs.Metrics.Close()

    l, err := link.AttachXDP(link.XDPOpts{
        Interface: 1, // lo (loopback) = index 1
        Program:   objs.Prog,
    })
    if err != nil {
        log.Fatalf("attach XDP: %v", err)
    }
}
```

```

defer l.Close()

numCPUs := runtime.NumCPU()
log.Printf("Monitoring packets across %d CPU cores ... ", numCPUs)

ticker := time.NewTicker(1 * time.Second)
defer ticker.Stop()

sig := make(chan os.Signal, 1)
signal.Notify(sig, os.Interrupt, syscall.SIGTERM)

for {
    select {
    case ←ticker.C:
        accept := readAndSumMetric(objs.Metrics, MetricAccept, numCPUs)
        drop := readAndSumMetric(objs.Metrics, MetricDrop, numCPUs)
        errCount := readAndSumMetric(objs.Metrics, MetricError, numCPUs)
        fmt.Printf("ACCEPT=%7d DROP=%7d ERROR=%7d (per second)\n",
            accept, drop, errCount)
    case ←sig:
        return
    }
}

// readAndSumMetric reads all per-CPU values for a given metric index and sums them.
// This is the correct way to read PERCPU_ARRAY maps in Go.
func readAndSumMetric(m *ebpf.Map, index int, numCPUs int) uint64 {
    key := uint32(index)
    // PERCPU_ARRAY lookup returns a slice of uint64, one per CPU
    values := make([]uint64, numCPUs)
    if err := m.Lookup(&key, &values); err != nil {
        return 0
    }
    var total uint64
    for _, v := range values {
        total += v
    }
    return total
}

```

9. Running the Lab

```
cd series/series-08/source-code
make
sudo ./loader
```

Generate packets in another terminal:

```
# Send 10,000 pings to generate packet load
ping -f -c 10000 127.0.0.1
```

Expected output:

```
ACCEPT= 14200 DROP=      0 ERROR=      0 (per second)
ACCEPT= 15800 DROP=      0 ERROR=      0 (per second)
ACCEPT= 16100 DROP=      0 ERROR=      0 (per second)
```

To see per-CPU breakdown, modify the Go code to print `values[cpu]` for each CPU.

10. Debugging

```
# Verify per-CPU map is loaded
sudo bpftool map list | grep metrics

# Dump all per-CPU values
sudo bpftool map lookup id <ID> key 0 0 0 0
# Output: per-cpu values for index 0 (accept counter)
# [cpu 0]: value: 0x38d0 (14544)
# [cpu 1]: value: 0x2200 (8704)
# [cpu 2]: value: 0x5200 (20992)
# [cpu 3]: value: 0x1d00 (7424)

# Attach XDP to a real NIC (replace lo with eth0)
sudo ip link set dev eth0 xdp object main.bpf.o section xdp
```

11. Common Mistakes

Mistake: Using atomic ops on per-CPU maps

```
// WRONG – unnecessary and slower
__sync_fetch_and_add(count, 1); // atomic add – not needed for per-CPU
```

Per-CPU maps guarantee that the BPF program running on CPU N only writes to CPU N's copy. No other CPU can concurrently write to the same copy. Atomics are unnecessary overhead.

Mistake: Incorrect per-CPU lookup in Go

```
// WRONG – treats per-CPU value as single u64
var val uint64
m.Lookup(&key, &val) // reads only CPU 0's value!
```

```
// CORRECT — allocate a slice of one uint64 per CPU
values := make([]uint64, runtime.NumCPU())
m.Lookup(&key, &values) // reads all CPUs
```

12. Performance Analysis

Metric	ARRAY (shared, locked)	PERCPU_ARRAY
Update latency per core	60–100ns (cross-cache)	~1ns (local write)
Throughput at 8 cores	~2–3 Mpps (contended)	~14+ Mpps (linear)
Memory per counter	8 bytes	8B × num_cpus
User-space aggregation	Direct read	Sum all CPU values

13. Best Practices

- **Never use atomics on per-CPU maps:** They are already exclusive per-CPU — atomics add overhead with no benefit.
- **Use per-CPU for all high-frequency counters in XDP/TC programs:** If your program fires more than 100K times/sec per core, use per-CPU.
- **Combine with PERCPU_HASH for per-source-IP rate limiting:** Each CPU maintains an independent count for each source IP; user-space aggregates periodically.

14. Summary

- CPU cache-line bouncing destroys performance when multiple cores update a shared counter.
- `BPF_MAP_TYPE_PERCPU_ARRAY` allocates one independent counter copy per CPU core.
- XDP programs write to their local CPU copy with ~1ns latency and zero contention.
- User-space must sum all CPU values to get the total — this is the correct aggregation pattern.
- This is the metrics architecture used by Meta's Katran to process billions of packets per second.

15. Further Reading

- **Meta Katran Architecture:** [Katran: A High-Performance Load Balancer](#)
- **CPU Cache and MESI Protocol:** [What Every Programmer Should Know About Memory](#)
- **BPF Per-CPU Maps:** [Linux BPF Per-CPU Map Internals](#)

Technical Deep-Dive Q&A: Per-CPU Maps for Lockless Metrics

1. Beginner Level

Q1. Why does updating a standard `BPF_MAP_TYPE_ARRAY` from multiple CPU cores simultaneously cause performance issues?

Answer: Updating a shared array map from multiple cores triggers **cache-line bouncing** via the CPU's cache coherence protocol (MESI). When CPU 0 writes to the map:

1. It must acquire the cache line containing the target memory in *Modified* state.
2. It sends invalidation requests to all other CPUs.
3. If CPU 1 is concurrently writing to the same map, its copy of the cache line is invalidated, forcing it to fetch the line back from memory or another CPU cache.
4. This invalidation loop (MESI bouncing) creates a massive inter-core interconnect bottleneck, causing instruction pipelines to stall. At 10Gbps line rates (XDP), this can drop throughput by up to 80%.

Q2. How does `BPF_MAP_TYPE_PERCPU_ARRAY` solve cache-line bouncing?

Answer: Instead of allocating a single shared memory block, the kernel allocates **N independent copies of the map values**, where N is the number of CPUs on the system.

- When XDP runs on CPU K, it looks up and updates the value in CPU K's private copy of the map.
 - Because each CPU accesses its own cache-aligned memory, there is no cross-core cache invalidation or MESI protocol overhead.
 - Updates become extremely cheap (~1ns local memory write), allowing linear performance scaling with the number of CPU cores.
-

2. Intermediate Level

Q3. Why do we not need atomic operations (like `__sync_fetch_and_add()`) when updating a per-CPU map?

Answer: In eBPF, XDP and TC programs running on CPU N cannot be preempted by other execution threads on CPU N. Since each CPU core has its own private map instance, and CPU N is the only core writing to its specific map copy, there is no race condition between different CPU cores. Consequently, plain memory operations (`*value += 1`) are completely thread-safe on a per-core level. This avoids the cost of atomic CPU instructions (`lock add`), which can save significant CPU cycles.

Q4. Walk through the memory allocation layout of a `BPF_MAP_TYPE_PERCPU_ARRAY`. How is alignment handled?

Answer: When you create a `BPF_MAP_TYPE_PERCPU_ARRAY` with a value size of 8 bytes (e.g., `__u64`):

1. The kernel does not allocate them contiguously. Instead, it rounds up the value size to match the system's cache-line alignment or a page boundary (typically 64 bytes or 128 bytes depending on hardware).
 2. It allocates N distinct, aligned buffers.
 3. Accessing index `i` on CPU `k` translates to accessing memory address `BaseAddress + (k * AlignedMapValueSize) + (i * ElementSize)`.
 4. This ensures that no two CPUs' metrics fall on the same 64-byte cache line, avoiding false sharing.
-

3. Advanced Level

Q5. How does Go retrieve and aggregate data from a `PERCPU_ARRAY` map?

Answer: In Go (using github.com/cilium/ebpf), a lookup call on a per-CPU map requires a slice containing one element per possible CPU:

```
// Create a slice with length equal to the number of CPUs
values := make([]uint64, runtime.NumCPU())
err := map.Lookup(&key, &values)
```

The kernel copies all N per-CPU values into this slice. User-space must then iterate over this slice and sum the values to compute the global metric.

4. System Design

Q6. Design a lockless packet counter for a high-performance DDoS mitigation system processing 40M packets/sec.

Answer:

- **BPF Layer:** Use XDP attached in driver mode (native XDP). Define a `BPF_MAP_TYPE_PERCPU_ARRAY` metrics map to count packets matching various classifications (TCP, UDP, ICMP, Dropped, etc.).
- **CPU Tuning:** Pin NIC receive queues to specific CPUs using RSS (Receive Side Scaling) and irqbalance settings.
- **Go Agent:** Query the per-CPU map using a ticker (every 1s). Fetch all CPU counts, sum them, and export them to Prometheus. Keep the BPF metrics path completely lock-free.

5. Troubleshooting

Q7. Why does my Go loader fail with `invalid argument` when doing a Lookup on a per-CPU map?

Answer: This usually happens because the length of the slice passed to `Lookup` does not match the number of possible CPUs expected by the kernel.

- `runtime.NumCPU()` returns the number of *online* CPUs, which might be lower than the *possible* CPUs if CPU hotplugging is enabled.
- In production, always retrieve the number of possible CPUs by reading `/sys/devices/system/cpu/possible` or calling the library's utility function, rather than using `runtime.NumCPU()`.

Hands-on Lab & Homework Assignments

1. Practical Exercises

Task 1: Check System Possible CPUs

In Go, learn how to check the possible CPU count:

```
import "github.com/cilium/ebpf/internal/sys"
// Use ebpf library's internal helpers to get the true possible CPU count
```

Task 2: Observe Per-CPU Metrics

Modify the Go loader to print the counter value for each individual CPU:

```
CPU 0: 4212 packets
CPU 1: 5391 packets
CPU 2: 12 packets
CPU 3: 8812 packets
```

Verify if packet processing is distributed evenly across all your CPU cores.

2. Coding Assignment: Per-CPU Protocol Breakdown

Objective: Build a per-CPU counter that categorizes packet sizes and protocols (TCP vs UDP vs ICMP).

Requirements:

1. Declare an enum for metrics:

```
enum {  
    TCP_PACKETS,  
    UDP_PACKETS,  
    ICMP_PACKETS,  
    OTHER_PACKETS,  
    METRIC_COUNT  
};
```

2. Parse IPPROTO in the XDP program and increment the correct bucket in a `BPF_MAP_TYPE_PERCPU_ARRAY`.
3. In Go, implement a CLI command `stats` that prints the sum of each protocol, as well as a raw breakdown of which CPU processed what percentage of the traffic.